



OS Development

4. Interrupts

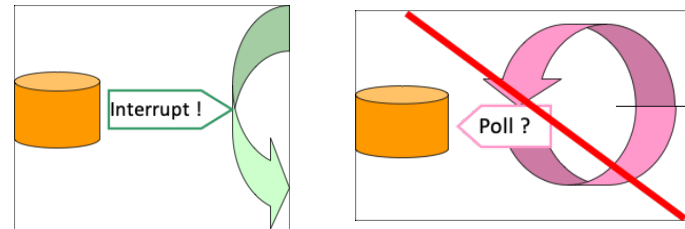
Prof. Dr. Michael Schöttner

Dr. Fabian Ruhland

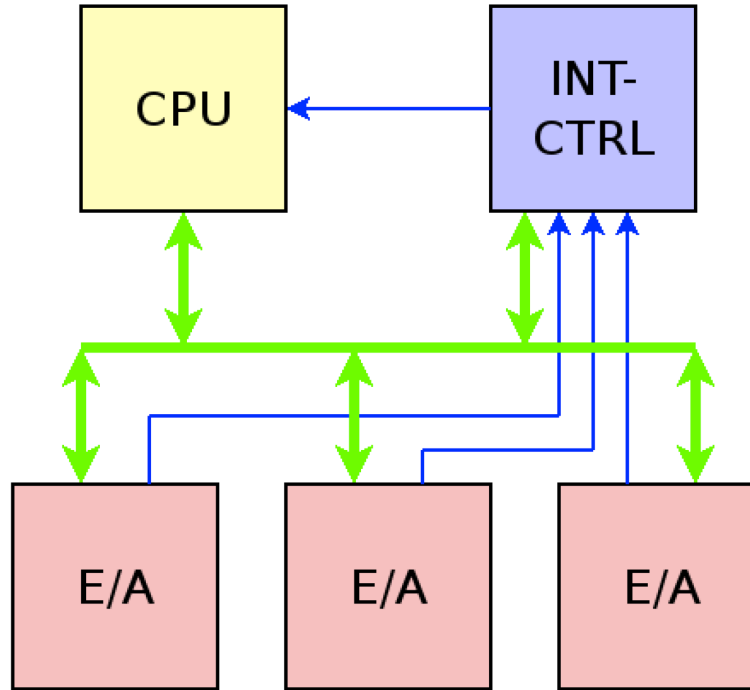
- Basics

- Interrupt Handling in x86_64
- Programmable Interrupt Controller
- Advanced Programmable Interrupt Controller
- Interrupt Handling in Operating Systems
- Summary

- Interrupts were introduced, to allow overlapping execution of input/output operations
- Instead of actively waiting for an I/O-operation to end, the affected thread is blocked and another thread is executed → This way, the CPU is used in a meaningful way
- Furthermore, interrupts form the basis of preemptive multithreading
 - Programme Interval Timer (PIT) generates periodic interrupts (see PC Speaker)
 - Preemptive multithreading is discussed (and implemented) later



Interrupt Hardware: Single Core



■ Background

- Multiple interrupts may be signaled at the same time → Which one is more important?
- While the CPU is handling an interrupt, further interrupts may be signaled

■ Priorization mechanism

- In software: The CPU only has a single IRQ (interrupt request) input
- In hardware: A priorization circuit assigns a priority to each device and only forwards the most important IRQ to the CPU
- Modern interrupt controllers allow dynamic priorities

■ Problem:

- Software should know which device has triggered an interrupt as fast as possible
 - We do not want to ask all devices manually

■ Solution: *Interrupt Vector*

- Assign a unique *vector number* to each interrupt, that is used as an index into a vector table
 - The vector number does not necessarily correlate with its priority
 - In practice, multiple device might have to share the same vector number (Shared Interrupts)
- Structure of the interrupt vector table depends on the CPU architecture
 - Oftentimes only contains function pointers

■ Problem:

- After executing an interrupt handler, the CPU must return to the thread context
- The interrupt handler should be "slid in" unnoticed

■ Solution: Store the system state

- By Hardware:
 - Only the essentials: e.g. *return address* and *flags register*
 - Restoring is done via a special instruction (e.g. `iret`, ...)
- By Software:
 - Because interrupts may occur any time, the handler must store and restore all registers it uses

■ Problem:

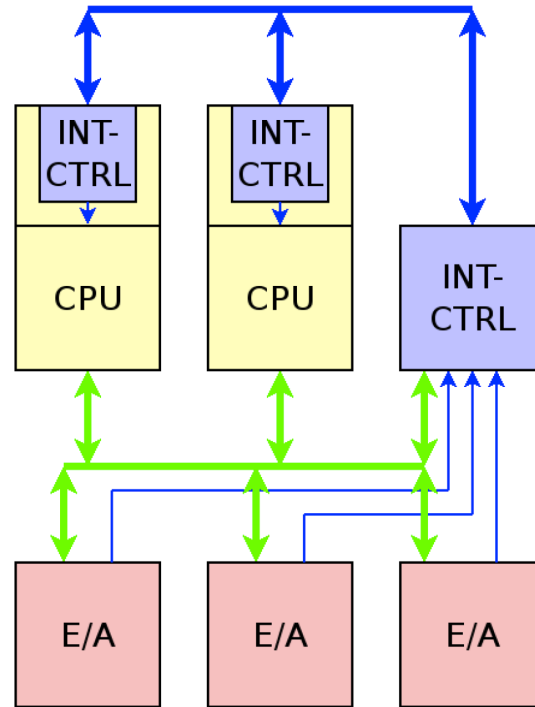
- To handle important events as fast as possible, interrupt handlers should also be interruptible
- Infinite nesting depth must be avoided

■ Solution:

- The CPU only allows interrupts with a higher priority
- Rarely implemented, since it introduces complexity and is usually not needed in practice

- Problems:
 - Which core handles an interrupt?
 - Handling *inter-processor interrupts* (IPI)

- Solution: Additional hardware for interrupt handling
 - Routing interrupts to CPU cores is controllable via software
 - Fixed or random assignment
 - Assignment with respect to the thread priorities on the respective CPU core



Danger: *Spurious Interrupts*

- Problem: Wrong interrupt request, because of
 - Hardware faults
 - Incorrectly programmed devices
- Solution:
 - Avoid hard- and software errors
 - "Defensive" programming in operating systems
 - Expect spurious interrupts

Danger: *Interrupt Storms*

■ Problem:

- Highly frequent interrupt request can paralyze an operating system
- Either caused by real interrupts, or the CPU is unable to cope with the I/O-load
- Can easily be confused with *Page Thrashing*

■ Solution (in the operating system):

- Detect interrupt storms
- Deactivate the affected device

- Basics
- Interrupt Handling in x86_64
- Programmable Interrupt Controller
- Advanced Programmable Interrupt Controller
- Interrupt Handling in Operating Systems
- Summary

■ External/Hardware interrupts

- Triggered by a device (e.g., timer)
- Come from the outside (from the CPU's point of view)

■ Internal/Software interrupts

- Triggered by the CPU itself
 - Exceptions (see next slides)
 - Assembly instruction `INT <nr>`
 - Used for *System Calls* (Linux, Windows NT, MacOS, MSDOS)

■ Fault

- Operating system can handle and recover from faults (e.g., Page Fault)
- CPU state and the address of the fault triggering instruction are stored

■ Trap

- Triggered by a special instruction (e.g., INT 3 - Breakpoint)
- Program can be continued

■ Abort

- Unrecoverable error
- Triggers are hard to localize exactly
- Cause a reboot (e.g., Double Fault)

- Interrupts and exceptions are identified via a *Vector Number*
- 0 – 31 are reserved for exceptions
- 32 – 255 are freely available

- Examples for exceptions:

Vector	Exception	Vector	Exception
0	Division by 0	11	Segment not present
1	Debug	12	Stack overflow
2	NMI	13	General protection fault
3	Break	14	Page fault
4	Overflow	16	-
5	Bounds range exceeded	18	Machine check
6	Illegal instruction	...	...
8	Double fault		

- Interrupt Descriptor Table (IDT)
- IDT start address is stored in IDTR (1 per core)
- 255 entries map each vector to an address (function pointer)
 - This function is the *Interrupt Handler / Interrupt Service Routine / ...*
 - 8 byte per entry (either *Interrupt Gate* or *Trap Gate*)
- **Interrupt-Gate**: Describes an interrupt handler
 - When this entry is used, all interrupts on the CPU core are masked (deactivated)
→ *Interrupt Enable* bit in RFLAGS is unset
- **Trap-Gate**: Describes a trap handler
 - Works the same as and interrupt gate, but without masking interrupts
 - For example used for system calls via software interrupts

Heinrich Heine
Universität Düsseldorf

- Interrupt/Trap Gate**

The diagram illustrates the structure of the Interrupt/Trap Gate, which is 32 bits long. It is divided into four main sections:

 - Reserved:** Bits 31 down to 12 (20 bits).
 - Offset 63..32:** Bits 31 down to 8 (24 bits).
 - Offset 31..16:** Bits 31 down to 16 (16 bits). This section is further divided into:
 - Offset 31..16 (bits 31-16)
 - P (bit 15)
 - DPL (bits 14-13)
 - 0 (bit 12)
 - TYPE (bits 11-8)
 - 0 0 0 (bits 7-5)
 - 0 0 (bits 4-3)
 - IST (bit 2)
 - Segment Selector and Offset 15..0:** Bits 31 down to 0 (32 bits). This section is further divided into:
 - Segment Selector (bits 31-16)
 - Offset 15..0 (bits 15-0)

Descriptor Privilege Level

Offset to procedure entry point

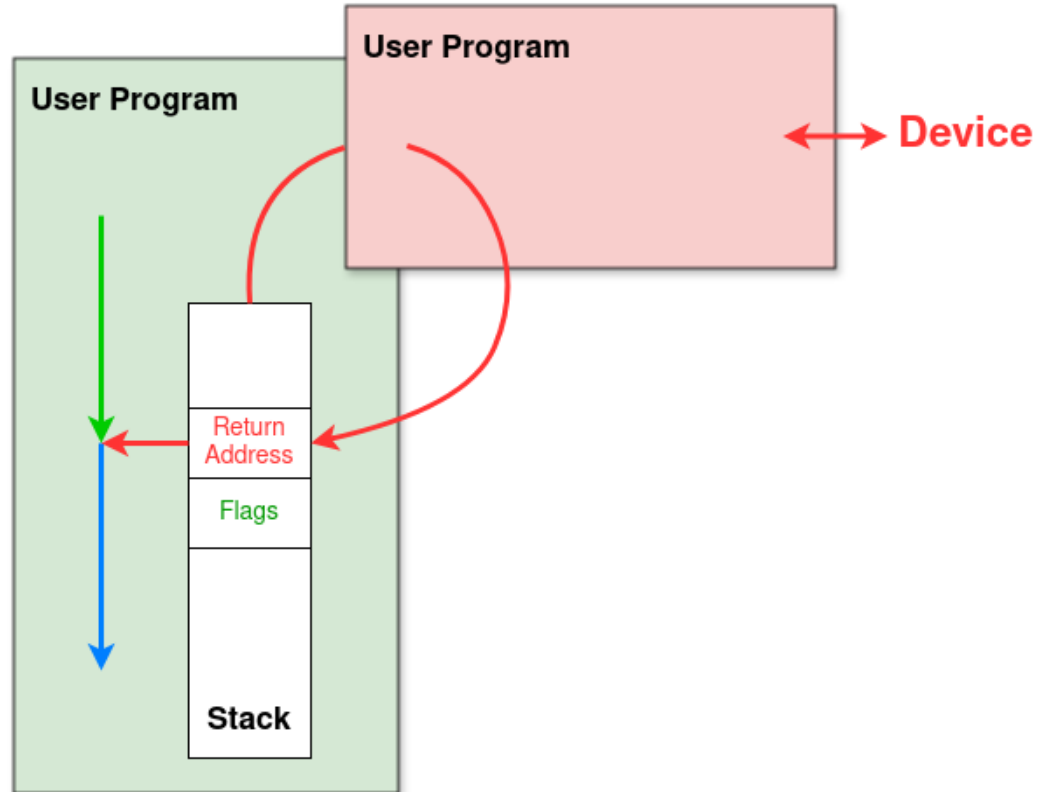
Segment Present flag

Segment Selector for destination code segment

Interrupt Stack Table

Stack Buildup during Interrupts

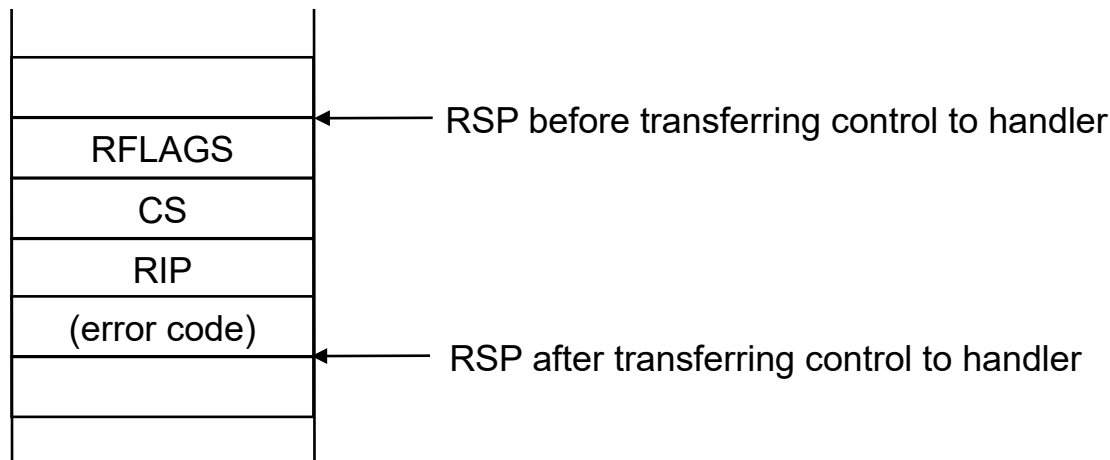
- Program is interrupted after execution of the last instruction



Stackup Buildup without Ring Change

- If the privilege level is not changed, the CPU does not switch the stack
- The interrupt handler operates on the stack of the interrupted thread
- This happens in HeineOS (all our code runs in ring 0)
- Only some exceptions push an error code

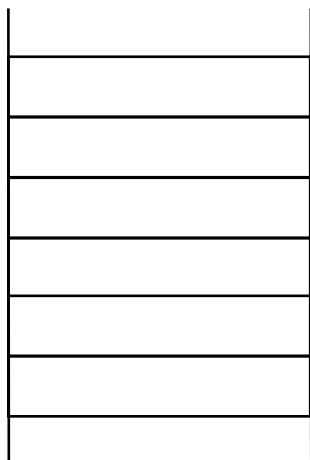
Stack of the interrupted thread



Stack Buildup with Ring Change

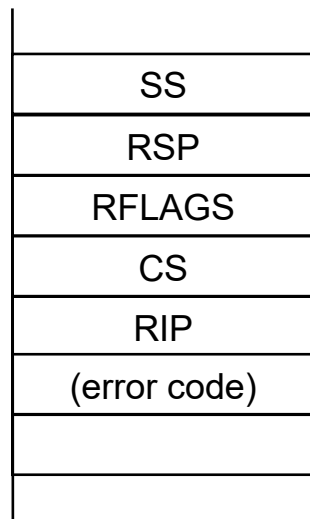
- New stack is read from *Task State Segment* (see last chapter)

Stack of the interrupted thread



RSP before transferring
control to handler

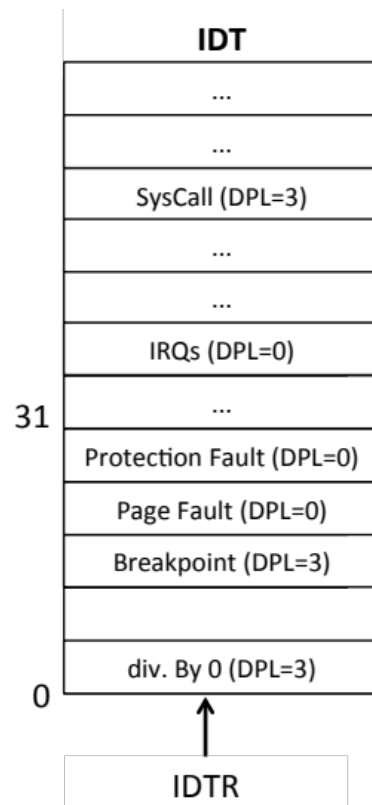
Handler Stack



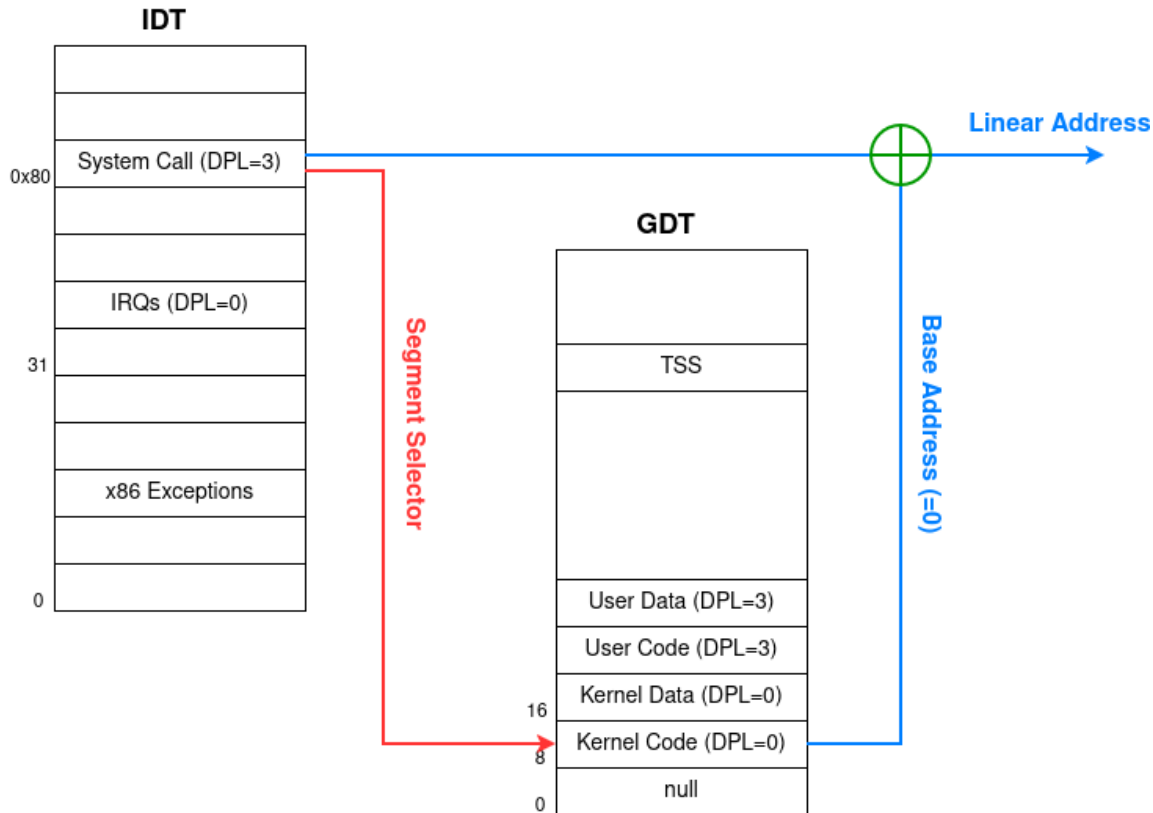
RSP after transferring
control to handler

IDT: Typical Example

- Entries 0-31 are reserved for x86 CPU exceptions
 - Either DPL=0 or DPL=3, depending on use case
- Entries for external interrupts
 - IRQs have DPL=0
- System calls via trap gate (Windows & Linux)
 - DPL=3
 - Replaced by SYSENTER/SYSEXIT in modern operating systems
→ faster, avoids interrupt overhead
- Since HeineOS runs completely in ring 0, we do not need trap gates



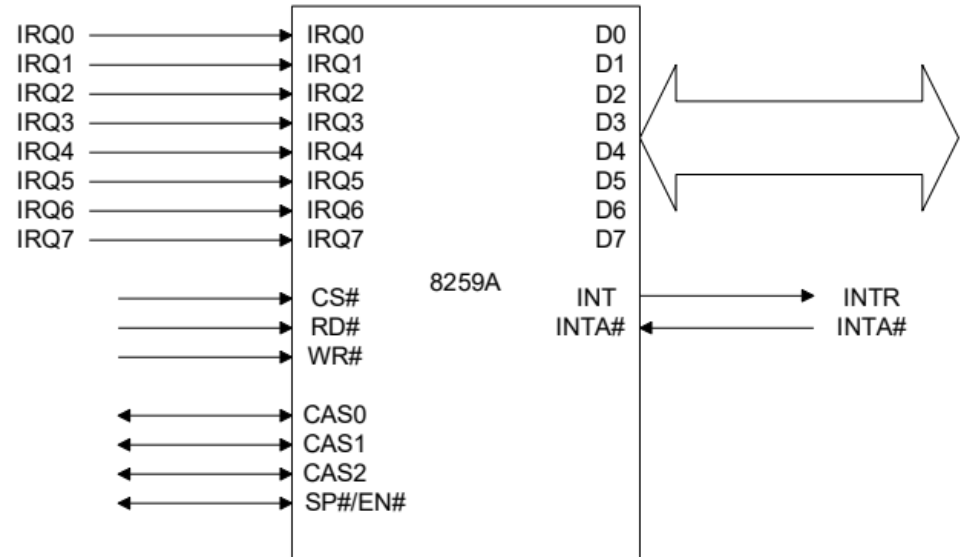
System Call via a Trap Gate



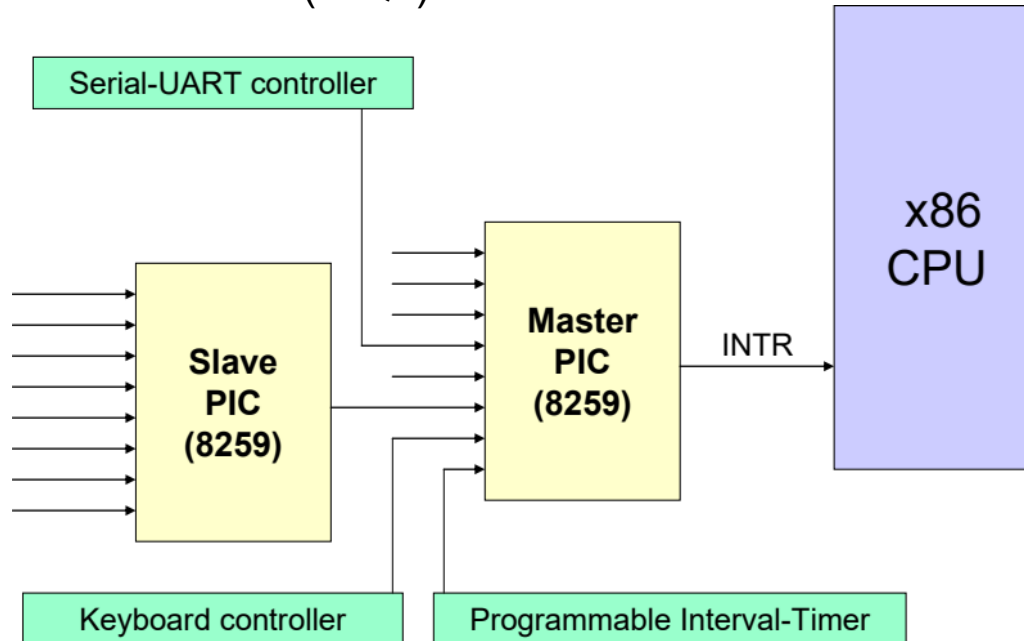
- Basics
- Interrupt Handling in x86_64
- Programmable Interrupt Controller
- Advanced Programmable Interrupt Controller
- Interrupt Handling in Operating Systems
- Summary

Interrupts in x86 CPUs

- Upto and including the i486 x86 CPUs only had a single input for IRQs
- External hardware was tasked with Priorization and vector number generation
 - Through a chip alled *PIC 8259A*
 - 8 interrupt inputs
 - 15 input by cascading two PICs



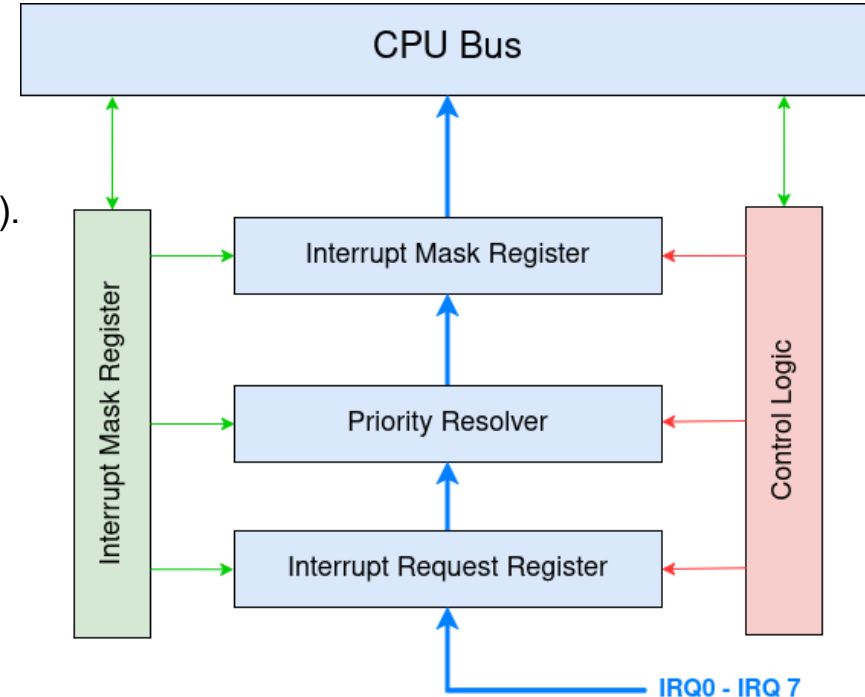
- Master controller can handle up to 8 slaves (CAS0..2 = Cascade Lines)
- In PCs only one slave is used (IRQ2)



- Typical IRQ assignment in PCs:
- Priorities
 - IRQ0 has highest priority
 - Default IRQ priorities: 0, 1, 8...15, 3... 7 (because of cascading)
 - Interrupts with a higher priority can interrupt ones with lower priorities (provided that the IE-bit (=Interrupt Enable) allows interrupts)

IRQ#	Device
0	Programmable Interval Timer
1	Keyboard Controller
2	Slave PIC
8	Real Time Clock
...	...
12	PS/2 Mouse
...	...
3	COM2
...	...

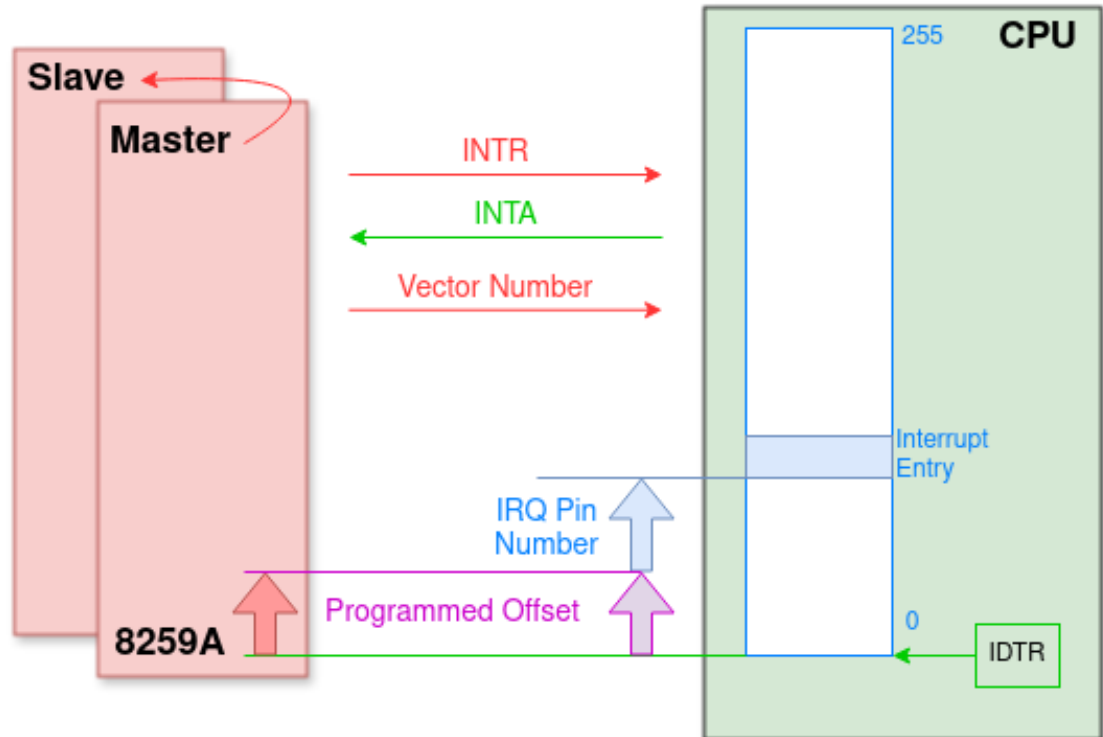
- 8 bit per register (1 bit per IRQ)
- Interrupt Mask Register (IMR)
 - Enable/Disable interrupts (0=enable, 1=masked).
- Interrupt Request Register (IRR):
 - Stores pending interrupts
- Priority Resolver:
 - Forwards IRQ with highest priority in IRR to ISR
- In Service Register (ISR):
 - Contains IRQs, that are currently being serviced



- IRQ line is activated → Corresponding bit in IRR is set
 - PIC sends INTR signal to CPU (slave PIC sends to master PIC instead)
 - CPU answers with INTA pulse, if IE bit is set in RFLAGS.
 - Most significant bit in IRR is unset and set in ISR.
 - CPU sends second INTA pulse
 - PIC puts vector pointer (8 bit = 3 bit IRQ + 5 bit offset) on data bus.
-
- Interrupt handler sends EOI to PIC when finished → deletes ISR bit
 - Remark: If interrupt came from slave PIC → also send EOI for IRQ2 to master PIC
 - EOI not necessary in HeineOS, since we use the PIC's automatic EOI mode

Interrupt Flow: PIC <=> CPU

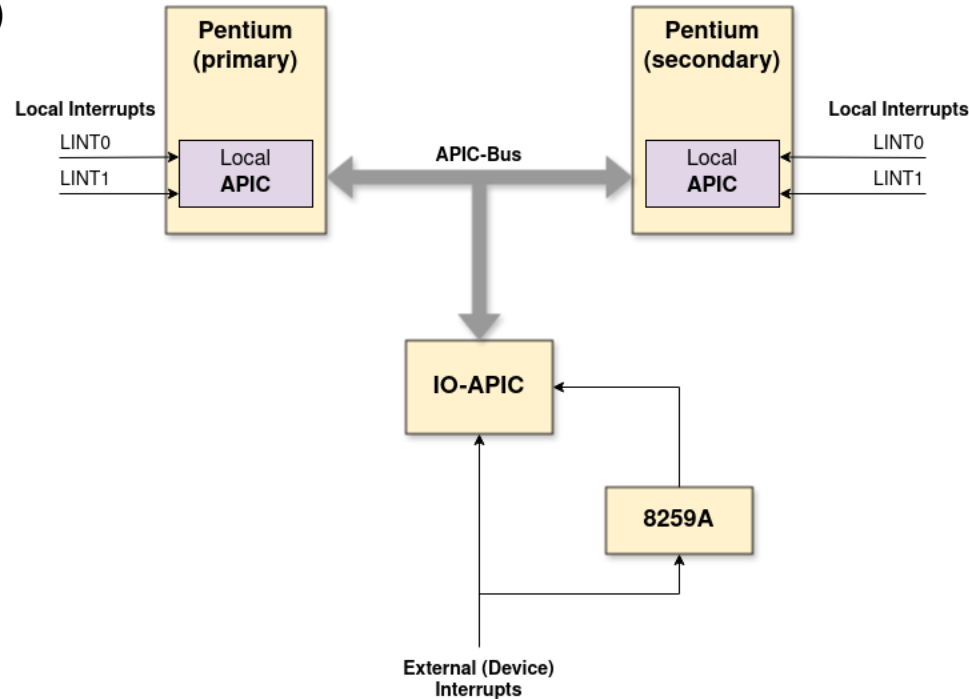
- Internal interrupts:
 - Fixed vector numbers 0 – 31
- External interrupts:
 - Vector = IRQ + Offset
 - Offset > 31 and IRQ != vector
 - Offset must map IRQs to vector number > 31



- Basics
- Interrupt Handling in x86_64
- Programmable Interrupt Controller
- Advanced Programmable Interrupt Controller
- Interrupt Handling in Operating Systems
- Summary

- APIC offers more interrupts than PIC and supports multicore and multiprocessor systems
 - Interrupts can be distributed to specific cores with respect to thread priorities
 - Inter-processor interrupts (IPI) between CPU cores (e.g, for bootstrapping other cores)
 - Backwards compatible with PIC (8259)
- If existent, the APIC is activated after booting, but runs in PIC compatible mode
 - APIC registers are found via *Model Specific Registers* (MSR)

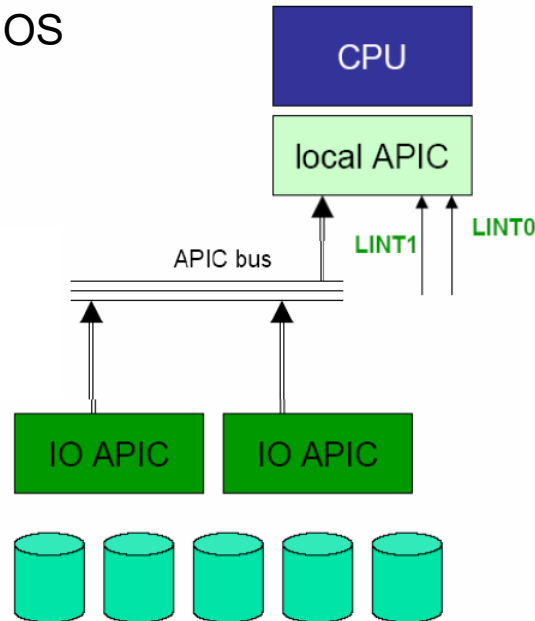
- An APIC interrupt system consists of a *Local APIC* per core and usually one *I/O-APIC* (may be more than one)



- Receives interrupt request as messages from the APIC bus
- Can additionally process two local interrupts directly
 - Via CPU pins: LINT0 and LINT1 (LINT = Local Interrupt)
 - Allow backwards compatibility with the PIC
- Local APICs are addressable via an ID
- Local APICs contain further function units
 - Builtin timer (dependent on system bus clock → divisible)
 - Interrupt command register
 - Used to send APIC messages → Needed for inter-processor interrupts (IPI)

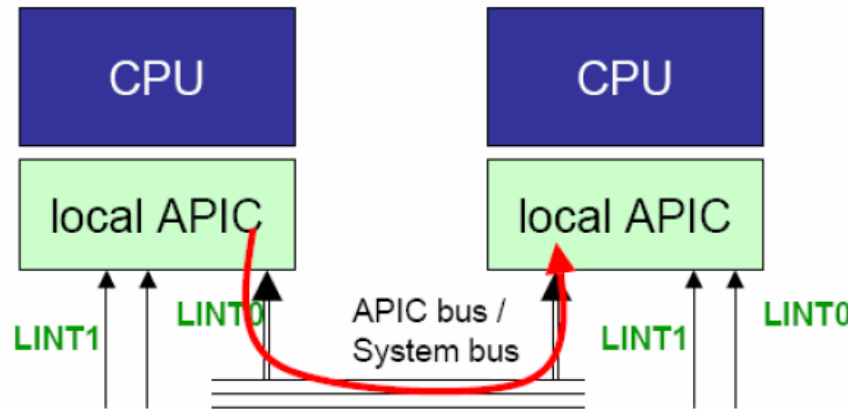
Interrupt Sources (1)

- External device interrupts are signaled by an IO-APIC via the APIC bus
- Default behaviour, once the APIC has been activated by the OS



■ IPI – Inter-Processor Interrupt

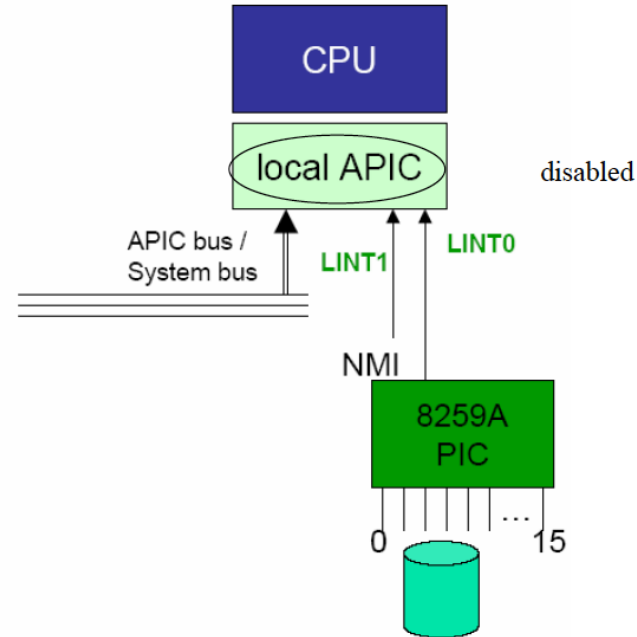
- Generated by another Local APIC → for multicore/multiprocessor systems
- Are sent as messages via the APIC
- For example use to coordinate startup/shutdown, or TLB flush after page tables have been altered



Interrupt Sources (3)

■ Local device interrupts:

- Via a CPU pin: LINT0, LINT1 (not via the APIC bus)
- Used for PIC compatibility mode, if the APIC has not been activated
- Examples:
 - Temperature sensor interrupt
 - 8259A (if existing)



Local APIC Registers

■ Task Priority Register (TPR)

- Stores thread priority
- Managed by the OS

■ Processor Priority Register

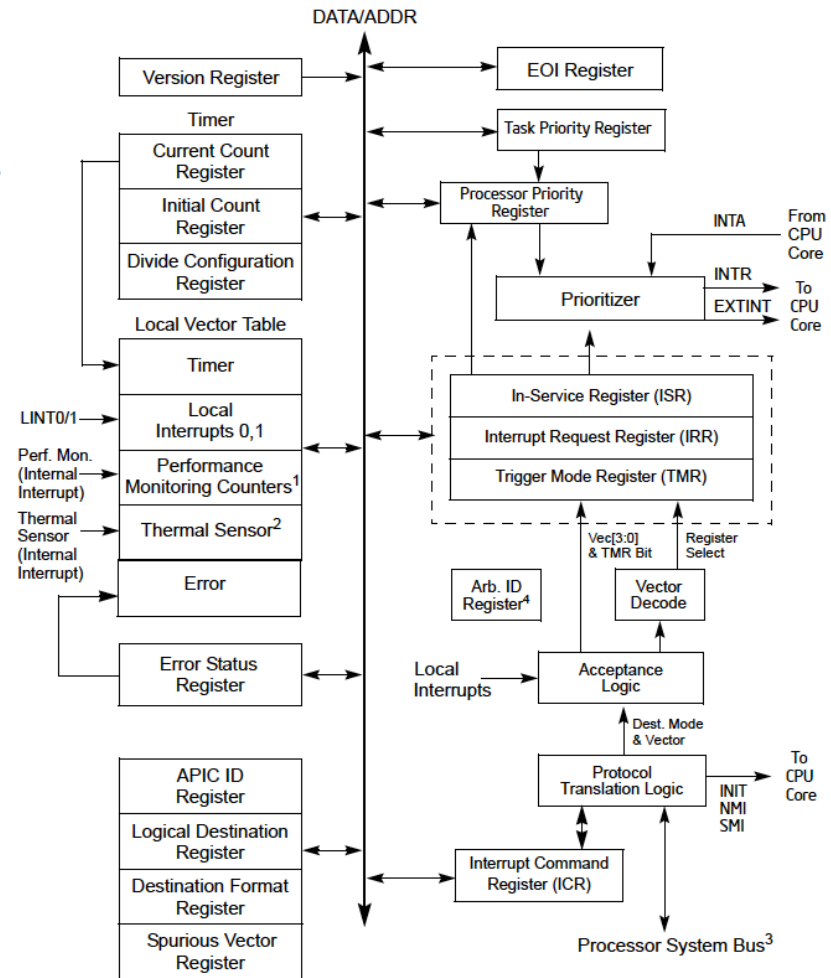
- Managed by the CPU
- Results from TPR & ISR

■ EOI Register

- End-Of-Interrupt
- Must be written before `iret`

■ Interrupt Command Register

- For Inter-processor interrupts



- Flat model addressing → Static interrupt delivery
 - No arbitration; Delivery to addressed Local APIC; Wait if necessary
- Dynamic interrupt delivery
 - After determining the group of addressed CPUs/APICs a „Lowest Priority Arbitration“ is performed
 - Goal: CPU that is currently processing the lowest priority task should handle the interrupt
 - Respect *Processor Priority Register* und *Task Priority Register*

- Nowadays typically integrated in the *Southbridge* of a PC chipset
- 24 interrupt inputs
 - Cyclic querying done by the chip
- Every interrupt input has a 64-bit entry in the *Interrupt Redirection Table*
 - Describes the interrupt signal
 - Determines the vector number and to which CPU core an interrupt is sent (used for generating the APIC bus message)
 - Interrupt Redirection Table (IRT) has 24 entries

Interrupt Redirection Table Entry

63:56	Destination Field (R/W)	8 bit target address → APIC ID (physical mode) or CPU group (logical mode)
55:17	Reserved	
16	Interrupt Mask (R/W)	Enable/Disable interrupts
15	Trigger Mode (R/W)	Edge/Level triggered interrupts
14	Remote IRR (RO)	
13	Interrupt Pin Polarity (R/W)	Signal polarity
12	Delivery Status (RO)	Interrupt message still in transit?
11	Destination Mode (R/W)	<i>Logical Mode</i> or <i>Physical Mode</i>
10:8	Delivery Mode (R/W)	000 – Fixed → Deliver signal to all target processors 001 – Lowest Priority → Deliver to CPU with lowest priority task 010 – SMI → System Management Interrupt 100 – NMI → Non-Maskable Interrupt 101 – INIT → Initialize (reset) target CPU 111 – ExtINT → Answer to 8259A
7:0	Interrupt Vector (R/W)	8 bit vector number (16-254)